The Journal of Open Source Software

StateCounter: Composable Counters with Unidirectional State Propagation for Enumerating Combinatorial Spaces

Zhihan Liu[1] **and Justin B. Kinney**[1¶]

1 Cold Spring Harbor Laboratory, Cold Spring Harbor, NY, USA ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

StateCounter is a Python library that provides composable counters with unidirectional state propagation for enumerating combinatorial spaces. The library enables users to declaratively define complex combinatorial structures using counter algebra operations—including Cartesian products, disjoint unions (stacks), slices, shuffles, samples, and splits—and then iterate through the resulting space while automatically tracking which component indices correspond to each enumerated state. StateCounter was developed to support the design of complex DNA sequence libraries but addresses a general problem that arises whenever random access to a combinatorial space is needed.

Statement of Need

Enumerating combinatorial spaces is a fundamental task in experimental design, machine learning, and scientific computing. Consider designing an experiment with multiple conditions: 3 treatments and 4 replicates yield 12 experimental samples. While nested loops make enumeration trivial, they fail when researchers need to:

- **Random access**: Given sample #7, determine its treatment and replicate indices
- **Shuffle**: Randomize sample order while tracking component indices
- **Sample**: Select a random subset while maintaining index correspondence
- **Split**: Divide into training and test sets with proper bookkeeping

The naive solution—manual index arithmetic using `divmod`—works for simple Cartesian products but does not compose. Real experimental designs often combine products *and* disjoint unions (e.g., 2 control samples plus a 3×4 treatment arm), and each additional operation (shuffle, slice, sample) requires rewriting all index logic. This approach is error-prone and tedious.

Existing Python tools provide partial solutions. The `itertools` module offers `product` and `chain` but provides no state tracking or composition—you cannot ask "given index 4, what are the component values?" after shuffling. NumPy's `unravel_index` and `ravel_multi_index` handle rectangular arrays but cannot represent non-rectangular structures like stacks. Neither approach supports composable operations with automatic index propagation.

StateCounter fills this gap by building a directed acyclic graph (DAG) of counters where setting the state of any derived counter automatically propagates to all ancestor counters. Users declare their combinatorial structure once, and StateCounter handles the index mathematics for every composed operation.

Functionality

StateCounter provides the following core operations:

Operation	Description
`product`	Cartesian product of counters ($\times$)
`stack`	Disjoint union where only one counter is active at a time ($+$)
`slice`	Select subset of states using Python slice syntax
`shuffle`	Randomly permute states with optional seed
`sample`	Sample states with or without replacement
`split`	Divide into multiple sub-counters by count or proportion
`repeat`	Cycle through states multiple times
`sync`	Keep multiple counters in lockstep
`interleave`	Alternate between counters in round-robin fashion

These operations compose freely. For example, a shuffled train/test split of a stacked product requires only:

```python
from statecounter import Counter, Manager, product, stack, shuffle, split

with Manager():
    control = Counter(num_states=2, name='control')
    treatment = Counter(num_states=3, name='treatment')
    replicate = Counter(num_states=4, name='replicate')

    samples = stack([control, product([treatment, replicate])])
    train, test = split(shuffle(samples, seed=42), [0.8, 0.2])

    for _ in test:
        print(f"control={control.state}, treatment={treatment.state}")
```

The key insight is **unidirectional state propagation**: iterating over any derived counter automatically updates all ancestor counters, with inactive counters (those not contributing to the current state) set to `None`. StateCounter also provides automatic conflict detection when a counter appears in multiple DAG branches with incompatible state requirements.

Research Applications

StateCounter was developed to support PoolParty (Kinney, 2026), a library for designing complex DNA sequence libraries for massively parallel reporter assays (MPRAs) and other high-throughput experiments. In this domain, researchers must enumerate variant libraries that combine multiple mutation types, positions, and backgrounds while maintaining the ability to shuffle, sample, and split the library for experimental and computational workflows.

Beyond molecular biology, StateCounter applies to any domain requiring structured enumeration:

- **Experimental design**: Randomizing treatment/control order while tracking conditions
- **Machine learning**: Creating stratified train/validation/test splits on combinatorial data
- **Hyperparameter search**: Enumerating parameter combinations with structured indices
- **Simulation studies**: Systematic exploration of parameter spaces with reproducible sampling

DRAFT

Acknowledgements

This work was supported by NIH grant R35 GM148235.

References

Kinney, J. B. (2026). *PoolParty: A python library for designing complex DNA sequence libraries*. https://github.com/jkinney/poolparty

DRAFT